



Sanjivani Rural Education Society's
Sanjivani College of Engineering, Kopargaon-423 603
(An Autonomous Institute, Affiliated to Savitribai Phule Pune University, Pune)
NAAC 'A' Grade Accredited, ISO 9001:2015 Certified

Department of Computer Engineering

(NBA Accredited)

Subject- Applied Machine Learning for Industry Solution
Assignment No-1

Dr. A. V. Brahmane
Assistant Professor

E-mail : brahmaneanilkumarcomp@sanjivani.org.in
Contact No: 91301 91301 Ext :145, 9922827812





Problem Defination

- Data Discretization and Data Normalization. Use any suitable dataset (e.g. heart dataset <https://www.kaggle.com/zhaoyingzhu/heartesv>).
- Perform following operations on given dataset suitable programming language.
 - Find standard deviation, variance of every numerical attribute.
 - Find covariance and perform Correlation analysis using Correlation coefficient.
 - How many independent features are present in the given dataset?
 - Can we identify unwanted features?
 - Perform the data discretization using equi frequency binning method on age attribute
 - Normalize RestBP, chol, and MaxHR attributes (considering above dataset) using min-max normalization, Z-score normalization, and decimal scaling normalization.



1. Load Dataset and Preprocess

- Before performing any operation, let's load the dataset and preprocess it.



Step 1: Import Necessary Libraries

```
python
```

```
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
import seaborn as sns
```



Step 2: Load Dataset

```
python
```

```
# Load the dataset (Update the path if needed)
```

```
df = pd.read_csv("heart.csv")
```

```
# Display first few rows
```

```
df.head()
```



Step 3: Check Missing Values

python

```
# Check if there are any missing values  
df.isnull().sum()
```

- If there are missing values, handle them using imputation techniques (mean/median/mode) or remove those rows.



2. Standard Deviation and Variance Calculation

- **Variance (σ^2)**
- Variance measures the spread of data points around the **mean**.
- A **high variance** means that the data points are spread out widely, while a **low variance** means they are close to the mean.
- Formula for Variance:

$$\sigma^2 = \frac{\sum (x_i - \mu)^2}{N}$$

Where:

- x_i = Each data point
- μ = Mean (average) of the data
- N = Total number of data points

- **Standard Deviation (σ)**
- Standard deviation is simply the **square root** of the variance.
- It is measured in the same units as the data, making it easier to interpret.

$$\sigma = \sqrt{\sigma^2}$$



2. Why Do We Calculate Standard Deviation and Variance in Data Mining?

- **Importance in Data Mining**
- **Understanding Data Spread:** It helps in understanding how much the data deviates from the mean.
- **Feature Selection:** High variance features might be important, while low variance ones may not add much information.
- **Outlier Detection:** If a data point is far from the mean (i.e., beyond a few standard deviations), it could be an outlier.
- **Normalization & Scaling:** Standard deviation helps in standardizing features (e.g., Z-score normalization).



- **Importance in This Assignment**
- In this assignment, standard deviation and variance will help us analyze **numerical attributes** in the dataset.
- It will help in understanding how different attributes like **RestBP (blood pressure)**, **Chol (cholesterol)**, and **MaxHR (maximum heart rate)** vary across patients.
- Later, we use **normalization techniques** that rely on these values.



3. Example Calculation (Manual)

- Let's say we have the following **RestBP (Blood Pressure)** values:

Patient	RestBP
1	120
2	130
3	110
4	140
5	125

- Step 1: Calculate the Mean (μ)

$$\mu = \frac{120 + 130 + 110 + 140 + 125}{5} = \frac{625}{5} = 125$$



Step 2: Compute Each Deviation from the Mean

Patient	RestBP (x_i)	$x_i - \mu$	$(x_i - \mu)^2$
1	120	-5	25
2	130	5	25
3	110	-15	225
4	140	15	225
5	125	0	0

Step 3: Compute Variance (σ^2)

$$\sigma^2 = \frac{(25 + 25 + 225 + 225 + 0)}{5} = \frac{500}{5} = 100$$

Step 4: Compute Standard Deviation (σ)

$$\sigma = \sqrt{100} = 10$$

So, for **RestBP**, the variance is **100**, and the standard deviation is **10**.



```
python

import pandas as pd

# Load dataset
df = pd.read_csv("heart.csv")

# Compute standard deviation and variance
std_dev = df.std()
variance = df.var()

# Display results
print("Standard Deviation:\n", std_dev)
print("\nVariance:\n", variance)
```



Interpretation of Standard Deviation and Covariance

- If RestBP has high variance, it means patients' blood pressure varies significantly.
- If RestBP has low variance, it means most patients have similar blood pressure.
- Attributes with very low variance may be removed as they do not contribute much to decision-making.
- High standard deviation means data is spread out, while low standard deviation means data is concentrated near the mean.



3. Covariance and Correlation Analysis

- **Covariance (Measures Relationship Strength & Direction)**
- **Covariance** measures the **direction** of the relationship between two numerical attributes.
- A **positive covariance** means both attributes **increase or decrease together**.
- A **negative covariance** means one attribute **increases while the other decreases**.
- A **covariance close to zero** means the two attributes **are not related**.

Formula for Covariance:

$$\text{Cov}(X, Y) = \frac{\sum (X_i - \mu_X)(Y_i - \mu_Y)}{N}$$

Where:

- X_i, Y_i = Data points of attributes X and Y
- μ_X, μ_Y = Mean of X and Y
- N = Total number of data points



Correlation (Measures Relationship Strength)

- **Correlation** measures both **strength** and **direction** of the relationship between two attributes.
- It is **normalized**, meaning its values range from **-1 to +1**:
 - **+1** → Strong **positive** correlation (when one increases, the other increases).
 - **0** → No correlation (no relationship between attributes).
 - **-1** → Strong **negative** correlation (when one increases, the other decreases).

Formula for Correlation (Pearson's Correlation Coefficient r):

$$r = \frac{\text{Cov}(X, Y)}{\sigma_X \cdot \sigma_Y}$$

Where:

- σ_X = Standard deviation of X
- σ_Y = Standard deviation of Y
- r is always between -1 and 1.



Why Do We Calculate Covariance and Correlation in Data Mining?

- **Feature Selection:** Helps in identifying **redundant features** (if two features are highly correlated, one can be removed).
- **Multicollinearity Detection:** If independent variables are **highly correlated**, it can affect machine learning models.
- **Understanding Data Relationships:** Helps in understanding **how different attributes influence each other**.
- **Pattern Recognition:** Helps in discovering relationships between medical conditions (e.g., **blood pressure vs. heart rate**).



- **Importance in This Assignment**
- We analyze relationships between attributes like:
 - **RestBP (Blood Pressure) vs. Cholesterol** → Do people with high blood pressure have high cholesterol?
 - **Age vs. MaxHR (Maximum Heart Rate)** → Does age affect heart rate?
- If two features are highly correlated, one may be **redundant** and can be removed.



Example Calculation (Manual)

Let's say we have the following data:

Patient	RestBP (X)	Cholesterol (Y)
1	120	200
2	130	210
3	110	190
4	140	220
5	125	205

Step 1: Compute Means (μ_X, μ_Y)

$$\mu_X = \frac{120 + 130 + 110 + 140 + 125}{5} = 125$$

$$\mu_Y = \frac{200 + 210 + 190 + 220 + 205}{5} = 205$$

Step 2: Compute $(X_i - \mu_X)$ and $(Y_i - \mu_Y)$

Patient	X_i	Y_i	$X_i - \mu_X$	$Y_i - \mu_Y$	$(X_i - \mu_X)(Y_i - \mu_Y)$
1	120	200	-5	-5	25
2	130	210	5	5	25
3	110	190	-15	-15	225
4	140	220	15	15	225
5	125	205	0	0	0

Step 3: Compute Covariance

$$\text{Cov}(X, Y) = \frac{25 + 25 + 225 + 225 + 0}{5} = \frac{500}{5} = 100$$

- So, RestBP and Chol have a positive covariance (100), meaning they tend to increase together.



Step 4: Compute Standard Deviations

Using the formula:

$$\sigma_X = \sqrt{\frac{(X_i - \mu_X)^2}{N}} = 10$$

$$\sigma_Y = \sqrt{\frac{(Y_i - \mu_Y)^2}{N}} = 10$$

Step 5: Compute Correlation

$$r = \frac{\text{Cov}(X, Y)}{\sigma_X \cdot \sigma_Y} = \frac{100}{10 \times 10} = 1.0$$

So, RestBP and Chol are perfectly positively correlated.



```
python
```

```
# Compute covariance matrix
```

```
cov_matrix = df.cov()
```

```
# Compute correlation matrix
```

```
corr_matrix = df.corr()
```

```
# Display covariance and correlation
```

```
print("Covariance Matrix:\n", cov_matrix)
```

```
print("\nCorrelation Matrix:\n", corr_matrix)
```

```
# Heatmap for better visualization
```

```
plt.figure(figsize=(10, 6))
```

```
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm", fmt=".2f")
```

```
plt.title("Correlation Matrix")
```

```
plt.show()
```



4. Identify Independent Features and Unwanted Features

-
- **Independent Features:** Features that do not have strong correlation with other variables.
- **Unwanted Features:** Features that do not contribute significantly to prediction or are highly correlated with others



python

```
# Find highly correlated features (correlation > 0.85)
threshold = 0.85
high_corr_features = set()

for i in range(len(corr_matrix.columns)):
    for j in range(i):
        if abs(corr_matrix.iloc[i, j]) > threshold:
            colname = corr_matrix.columns[i]
            high_corr_features.add(colname)

print("Highly Correlated Features (Possibly Unwanted):", high_corr_features)

# Drop unwanted features if needed
df_reduced = df.drop(columns=high_corr_features)
```



Data Discretization using Equi-Frequency Binning

- **What is Data Discretization?**
- **Data Discretization** is the process of converting continuous numerical attributes into discrete categories or bins.
- It helps in simplifying data, reducing complexity, and improving the performance of machine learning algorithms.
- There are different methods of discretization, such as:
 - **Equal-width binning**
 - **Equal-frequency binning (Quantile-based binning)**
 - **Clustering-based discretization (e.g., k-means clustering)**



- **What is Equi-Frequency Binning?**
- **Equi-Frequency Binning (Quantile Binning)** divides the data into bins **such that each bin has approximately the same number of data points**.
- It is **different from Equal-Width Binning**, where bins have equal width but may contain an unequal number of data points.

Formula for Equi-Frequency Binning:

1. Sort the attribute values in ascending order.
2. Divide the sorted data into k bins, ensuring **each bin has approximately N/k values**, where:

N = Total number of data points, k = Number of bins

3. **Assign** each data point to a bin.



Why Do We Use Equi-Frequency Binning in Data Mining?

- **Advantages of Equi-Frequency Binning:**
- Helps in handling **skewed distributions** effectively.
- Reduces the impact of **outliers** (because bins have an equal number of points, extreme values do not distort the bins).
- Improves the **performance of classification and clustering** algorithms by simplifying data.
- Useful for **probability density estimation**.



- **Importance in This Assignment (Heart Dataset Example)**
- We discretize the **Age** attribute using **Equi-Frequency Binning** to group patients into age categories.
- Helps in **categorizing patients** into risk groups (e.g., young, middle-aged, elderly).
- Useful for **decision tree models**, where categorical features work better than continuous features.



4. Example Calculation (Manual Step-by-Step)

Dataset (Age Attribute)

Suppose we have the following **Age** values from our dataset:

$$\text{Age} = [25, 30, 35, 40, 45, 50, 55, 60, 65, 70]$$

Step 1: Sort the Data in Ascending Order

$$\text{Sorted Age} = [25, 30, 35, 40, 45, 50, 55, 60, 65, 70]$$

Step 2: Decide Number of Bins (k)

Let's assume we want to divide the **Age** attribute into 3 bins ($k = 3$).

Total number of values = 10

Each bin should have $N/k = 10/3 \approx 3$ values per bin.

Step 3: Divide Data into Equal Frequency Bins

- Bin 1: First 3 values → [25, 30, 35]
- Bin 2: Next 3 values → [40, 45, 50]
- Bin 3: Next 3 values → [55, 60, 65]
- Bin 4: Remaining 1 value → [70] (this bin is smaller)



Step 4: Assign Bin Labels

- To make the bins categorical, we can assign **labels**:
- Now, the **Age** attribute is **discretized into meaningful categories**.

Age	Bin Label
25	Young (Bin 1)
30	Young (Bin 1)
35	Young (Bin 1)
40	Middle-Aged (Bin 2)
45	Middle-Aged (Bin 2)
50	Middle-Aged (Bin 2)
55	Elderly (Bin 3)
60	Elderly (Bin 3)
65	Elderly (Bin 3)
70	Very Elderly (Bin 4)



```
import pandas as pd

# Sample data (simulating Age column from the Heart dataset)
age_data = {'Age': [25, 30, 35, 40, 45, 50, 55, 60, 65, 70]}
df = pd.DataFrame(age_data)

# Perform Equi-Frequency Binning into 3 bins
df['Age_Binned'] = pd.qcut(df['Age'], q=3, labels=['Young', 'Middle-Aged', 'Elderly'])

# Display the output
print(df)
```



Data Normalization

- **Data Normalization** is the process of transforming numerical attributes into a common scale **without distorting differences in the ranges of values**.
- Some machine learning algorithms (like KNN, SVM, and Neural Networks) **perform better when input features are on a similar scale**.
- It helps in making different attributes **comparable**.
- Normalization is useful when numerical attributes have **different units or ranges** (e.g., blood pressure in mmHg, cholesterol in mg/dL).



- **Removes bias** due to attributes having different scales.
 - **Speeds up learning algorithms** by ensuring gradient descent converges faster.
 - **Improves model performance** by reducing the impact of features with large magnitudes.
 - **Prepares data for machine learning models**, especially distance-based algorithms (KNN, clustering).
 - **Prevents numerical instability** in models like Neural Networks.
-
- **Importance in This Assignment (Heart Dataset Example)**
 - The attributes **RestBP (Resting Blood Pressure)**, **Chol (Cholesterol)**, and **MaxHR (Maximum Heart Rate)** have different units and ranges.
 - Normalizing these attributes ensures **fair comparison** and improves **model accuracy**.
 - It helps in **detecting anomalies** (e.g., extremely high cholesterol or blood pressure).



Data Normalization

- Normalization scales numeric data into a standard range.
- Three common methods:
- **Min-Max Normalization:** Scales values between 0 and 1.
- **Z-score Normalization:** Centers data around mean (standard deviation = 1).
- **Decimal Scaling:** Moves the decimal point based on the maximum absolute value.



Min-Max Normalization

Formula:

$$X' = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

- Transforms data to a **fixed range** (usually 0 to 1 or -1 to 1).
- Retains **relationships between values** but compresses the range.

Example Calculation for RestBP:

Assume the following values for RestBP:

$$\text{RestBP} = [120, 140, 160, 180, 200]$$

Find **min** and **max**:

$$X_{\min} = 120, \quad X_{\max} = 200$$



RestBP	Normalized RestBP
120	$(120-120)/(200-120)=0$ $(120 - 120) / (200 - 120) = 0$ $(120-120)/(200-120)=0$
140	$(140-120)/(200-120)=0.25$ $(140 - 120) / (200 - 120) = 0.25$ $(140-120)/(200-120)=0.25$
160	$(160-120)/(200-120)=0.50$ $(160 - 120) / (200 - 120) = 0.50$ $(160-120)/(200-120)=0.50$
180	$(180-120)/(200-120)=0.75$ $(180 - 120) / (200 - 120) = 0.75$ $(180-120)/(200-120)=0.75$
200	$(200-120)/(200-120)=1$ $(200 - 120) / (200 - 120) = 1$ $(200-120)/(200-120)=1$



Z-Score Normalization (Standardization)

Formula:

$$X' = \frac{X - \mu}{\sigma}$$

- Centers the data around mean 0 and variance 1.
- Useful when data follows a normal distribution.

Example Calculation for Cholesterol (Chol):

Assume:

$$\text{Chol} = [180, 220, 240, 260, 300]$$

Find **Mean** (μ) and **Standard Deviation** (σ):

$$\mu = \frac{180 + 220 + 240 + 260 + 300}{5} = 240$$

$$\sigma = \sqrt{\frac{(180 - 240)^2 + (220 - 240)^2 + (240 - 240)^2 + (260 - 240)^2 + (300 - 240)^2}{5}}$$

$$\sigma = 40$$



Chol	Z-score Normalized Chol
180	$(180-240)/40=-1.5$ $(180 - 240) / 40 = -1.5$ $(180-240)/40=-1.5$
220	$(220-240)/40=-0.5$ $(220 - 240) / 40 = -0.5$ $(220-240)/40=-0.5$
240	$(240-240)/40=0$ $(240 - 240) / 40 = 0$ $(240-240)/40=0$
260	$(260-240)/40=0.5$ $(260 - 240) / 40 = 0.5$ $(260-240)/40=0.5$
300	$(300-240)/40=1.5$ $(300 - 240) / 40 = 1.5$ $(300-240)/40=1.5$

After **Z-Score Normalization**, values are centered around **0**.



Decimal Scaling Normalization

Formula:

$$X' = \frac{X}{10^j}$$

where j is the smallest integer such that $\max(|X'|) < 1$.

Example Calculation for MaxHR:

Assume:

$$\text{MaxHR} = [100, 150, 200, 250, 300]$$

Find **max value**:

$$X_{\max} = 300$$

Choose $j = 3$ because $300 < 10^3$.



MaxHR	Decimal Scaled MaxHR
100	$100/103=0.1100 / 10^3 = 0.1100/103=0.1$
150	$150/103=0.15150 / 10^3 = 0.15150/103=0.15$
200	$200/103=0.2200 / 10^3 = 0.2200/103=0.2$
250	$250/103=0.25250 / 10^3 = 0.25250/103=0.25$
300	$300/103=0.3300 / 10^3 = 0.3300/103=0.3$

After **Decimal Scaling**, values are in the range **[-1,1]**

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import MinMaxScaler, StandardScaler

# Sample Data (Heart dataset attributes)
data = {
    'RestBP': [120, 140, 160, 180, 200],
    'Chol': [180, 220, 240, 260, 300],
    'MaxHR': [100, 150, 200, 250, 300]
}
df = pd.DataFrame(data)
```



```
# Min-Max Normalization
scaler_minmax = MinMaxScaler()
df_minmax = pd.DataFrame(scaler_minmax.fit_transform(df), columns=df.columns)

# Z-Score Normalization
scaler_zscore = StandardScaler()
df_zscore = pd.DataFrame(scaler_zscore.fit_transform(df), columns=df.columns)

# Decimal Scaling Normalization
df_decimal = df / (10 ** np.ceil(np.log10(df.abs().max())))

# Display Results
print("\nOriginal Data:\n", df)
print("\nMin-Max Normalized:\n", df_minmax)
print("\nZ-Score Normalized:\n", df_zscore)
print("\nDecimal Scaled:\n", df_decimal)
```



Conclusion

Operation	Explanation
Standard Deviation & Variance	Measures spread of numerical data
Covariance & Correlation	Shows relationship between features
Independent Features	Features that don't depend on other variables
Unwanted Features	Highly correlated features that can be removed
Data Discretization	Converts numerical data into categorical bins
Min-Max Normalization	Scales data between 0 and 1
Z-score Normalization	Centers data around mean with std deviation 1
Decimal Scaling	Adjusts data by shifting decimal point



•Thank you...